Report 2: UML Class Diagram

Library Management System (LMS)

Course / Institution	Kigali Independent University (ULK)
Assignment	UML Class Diagram Modelling Assignment
Instructor	Eric Maniraguha
Group Member 1	[Full Name] - Student ID: [_____]
Group Member 2	[Full Name] - Student ID: [_____]
Submission date	20 October 2026

1. Introduction

This report presents the UML class diagram of the Library Management System. The diagram contains **9 classes**, their attributes and methods, and the relationships between them with multiplicities. The diagram is on the next page (landscape). The editable file is *diagrams/uml_class_diagram.drawio*.

Notation: + public, - private. Generalization = hollow triangle; composition = filled diamond; aggregation = hollow diamond; association = plain line.

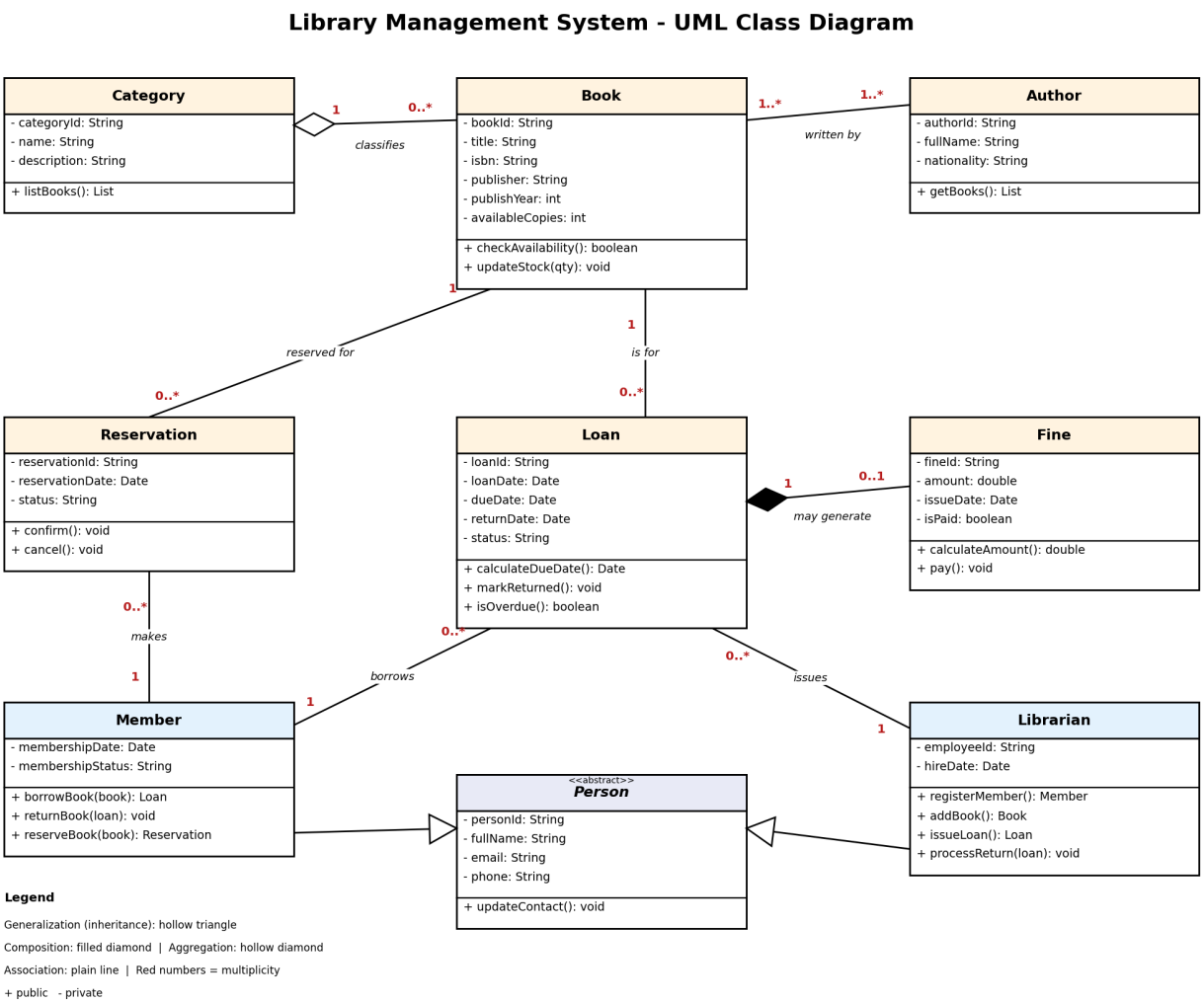


Figure 1: UML class diagram of the Library Management System

2. Classes, Attributes and Methods

Person (abstract)

Attributes	Methods
- personId: String - fullName: String - email: String - phone: String	+ updateContact(): void

Member

Attributes	Methods
- membershipDate: Date - membershipStatus: String	+ borrowBook(book): Loan + returnBook(loan): void + reserveBook(book): Reservation

Librarian

Attributes	Methods
- employeeId: String - hireDate: Date	+ registerMember(): Member + addBook(): Book + issueLoan(): Loan + processReturn(loan): void

Book

Attributes	Methods
- bookId: String - title: String - isbn: String - publisher: String - publishYear: int - availableCopies: int	+ checkAvailability(): boolean + updateStock(qty): void

Author

Attributes	Methods
- authorId: String - fullName: String - nationality: String	+ getBooks(): List

Category

Attributes	Methods
- categoryId: String - name: String - description: String	+ listBooks(): List

Loan

Attributes	Methods
- loanId: String - loanDate: Date - dueDate: Date - returnDate: Date - status: String	+ calculateDueDate(): Date + markReturned(): void + isOverdue(): boolean

Reservation

Attributes	Methods
- reservationId: String - reservationDate: Date - status: String	+ confirm(): void + cancel(): void

Fine

Attributes	Methods
- fineId: String - amount: double - issueDate: Date - isPaid: boolean	+ calculateAmount(): double + pay(): void

3. Relationships and Multiplicities

#	Relationship	Type	Multiplicity	Meaning
1	Member - Loan	Association	1 to 0..*	A member borrows zero or many loans; each loan belongs to exactly one member.
2	Member - Reservation	Association	1 to 0..*	A member can make many reservations; each reservation is made by one member.
3	Librarian - Loan	Association	1 to 0..*	A librarian issues many loans; each loan is issued by one librarian.
4	Loan - Book	Association	0..* to 1	Each loan is for one book; a book can appear in many loans over time.
5	Reservation - Book	Association	0..* to 1	Each reservation is for one book; a book can be reserved many times.
6	Book - Author	Association (many-to-many)	1..* to 1..*	A book has at least one author; an author can write many books.
7	Category - Book	Aggregation	1 to 0..*	A category groups many books; books continue to exist if the category is removed.
8	Loan - Fine	Composition	1 to 0..1	A late loan may generate one fine; the fine cannot exist without its loan.
9	Person - Member / Librarian	Generalization	-	Member and Librarian inherit the common attributes and methods of Person.

4. Explanation of the Design

• **Inheritance:** Person is an abstract class because the system never creates a plain 'person'. Member and Librarian reuse its attributes, which avoids repetition.

• **Loan as a separate class:** the relation between Member and Book is many-to-many (a member borrows many books, a book is borrowed by many members). It is resolved with the Loan class, which also carries its own data (dates and status).

• **Reservation** works like Loan: it links a Member to a Book and stores the reservation date and status.

• **Composition (Loan-Fine):** a Fine has no meaning without its Loan, so it is destroyed with it. The multiplicity 0..1 shows that not every loan produces a fine.

• **Aggregation (Category-Book):** a Category is a whole made of Books, but books remain valid objects even without a category.

• **Many-to-many (Book-Author):** a book can have several authors and an author can write several books.

5. Conclusion

The class model covers the main information of a library and shows clear relationships with correct multiplicities. It can be used directly as a base for database design and for implementing the system in the future.